

# Análisis del sistema — qué anda mal y qué se puede mejorar

---

**Alcance:** backend `gestion_rrhh/` (Django + DRF + Postgres) y frontend `frontend/` (canvas de Claude Design + capa de integración `ceibo-api.js` + `build.py`). **Fecha:** 2026-07-14 · rama `fase-0-verificada`.  
**Última actualización:** 2026-07-15 — la sección **B (correctitud e integridad)** **está resuelta**; sus hallazgos se quitaron de este documento (ver "B. Correctitud..." abajo).

Cada hallazgo tiene un ID (A = seguridad, B = correctitud/integridad, C = robustez, D = mejoras) y referencia a archivo y línea. Al final hay una tabla de prioridades.

---

## Lo que está bien (contexto)

---

Antes de los problemas, vale decir que la base es sólida y mejor que el promedio de un MVP:

- Arquitectura limpia y consistente: views flacas → services (escritura, transaccional) → selectors (lectura scopeada por rol). Se cumple en todas las apps.
  - Reglas de dominio bien pensadas y documentadas en el código (R1, R10, R11, RP1–RP7, cadena de prórrogas con vigencia calculada y nunca persistida).
  - Constraints reales en la base (UNIQUE parcial para R1, UNIQUE de documento vigente) además de la validación amigable en el service.
  - JWT con rotación y blacklist, throttling, manejador de errores JSON uniforme, OpenAPI como contrato, paginación con tope.
  - Tests de API en todas las apps; seeds reproducibles; docker compose con healthcheck.
  - El pipeline del frontend (design intake → `build.py` con anclas que fallan ruidosamente → `dist/`) es ingenioso y está bien protegido contra pisadas.
- 

## A. Seguridad

---

### A1 — Credenciales de admin hardcodeadas en el frontend

`frontend/integration/ceibo-api.js:14–18`

```
var CONFIG = { API: "...", USER: "admin", PASS: "Clave-Segura-123" };
```

Cualquiera que abra el front (o el repo) tiene la clave de un superusuario, y la app no tiene pantalla de login: todo visitante ES admin. Todo el sistema de roles del backend (Admin/RRHH/Supervisor/Empleado) queda sin efecto en la práctica. Es un atajo de MVP conocido, pero es **lo primero** a resolver antes de cualquier

deploy o de dar acceso a terceros. Además la clave ya quedó en el historial de git: cuando exista un entorno real, rotarla.

**Fix:** pantalla de login que pida usuario/clave contra `/auth/token/` y guarde el refresh en memoria (o `sessionStorage`), y eliminar `USER / PASS` de `CONFIG`.

## A2 — Fuga de scope: cualquier autenticado ve documentos de cualquier empleado ●

`gestion_rrhh/apps/empleados/api/views.py:73-78`

La acción `documentos` (GET) tiene permiso `IsAuthenticated`, pero resuelve el empleado con `get_object_or_404(Empleado, pk=pk)` **sin pasar por el selector**. El scoping por rol ("el Empleado ve solo su ficha") se aplica en `list / retrieve` vía `get_queryset()`, pero acá se lo saltea: un usuario con rol Empleado puede pedir `GET /api/v1/empleados/{cualquier_id}/documentos/` y ver los documentos (número, vencimientos, observaciones) de cualquier otra persona.

**Fix:** `empleado = get_object_or_404(self.get_queryset(), pk=pk)` en la rama GET (las acciones de escritura ya exigen RRHH/Admin, ahí no hay fuga).

## A3 — PII expuesta a roles que no deberían verla ●

`gestion_rrhh/apps/empleados/api/serializers.py:36-65` vs. la promesa de `gestion_rrhh/apps/empleados/models.py:52` ("El PII (dni/cuil) se expone solo a RRHH/Admin").

`EmpleadoSerializer` es único para todos los roles: un **Supervisor** (que ve toda la dotación) recibe dni, cuil, dirección, teléfono, contacto de emergencia, obra social y observaciones de todos. El modelo documenta otra intención.

**Fix:** serializer reducido (sin PII) para Supervisor, o campos condicionales por rol en `to_representation`.

## A4 — `SECRET_KEY` con default inseguro alcanza también a prod ●

`gestion_rrhh/config/settings/base.py:17` y `prod.py`

`SECRET_KEY = env("SECRET_KEY", default="insegura-solo-para-dev")` vive en `base.py` y `prod.py` no lo redefine: si en prod falta la variable de entorno, el proceso **arranca igual** con la clave insegura (firmando JWTs con ella).

**Fix:** en `prod.py`, `SECRET_KEY = env("SECRET_KEY")` sin default, para que prod explote al arrancar si falta.

---

## B. Correctitud e integridad de datos ✅ resuelta (2026-07-15)

Los seis hallazgos (B1–B6) están arreglados y verificados contra Postgres. Se quitan de este documento; quedan acá solo el registro de qué se hizo y las consecuencias que sobreviven al fix. Detalle en el commit y en los tests que los cubren.

| I<br>D | Era                                                                                                                 | Quedó                                                                                                                      |
|--------|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| B<br>1 | Filtro empresa+estado usaba un JOIN por <code>.filter()</code> y cada condición la satisfacía una relación distinta | <code>empresa / sector / estado</code> se combinan en un solo <code>.filter()</code> — <code>empleados/selectors.py</code> |
| B<br>2 | El PATCH podía romper la cadena de prórrogas ( <code>fecha_desde</code> , <code>tipo_novedad</code> )               | <code>actualizar_novedad</code> los rechaza si <code>novedad.es_prorroga</code>                                            |
| B<br>3 | El no-solapamiento solo vivía en Python (SELECT+INSERT, carrera abierta)                                            | Lock pesimista sobre el empleado + <code>ExclusionConstraint</code> con <code>btree_gist</code>                            |
| B<br>4 | Un documento cargado no se podía corregir ni renovar                                                                | PATCH / DELETE de <code>/empleados/{id}/documentos/{doc_id}/</code>                                                        |
| B<br>5 | El legajo lo calculaba el navegador con <code>max+1</code>                                                          | Lo asigna el backend con advisory lock; el cliente no puede elegirlo                                                       |
| B<br>6 | <code>todayISO()</code> devolvía la fecha UTC (de noche, mañana)                                                    | Se arma con getters locales                                                                                                |

**Un bug extra, no detectado en el análisis original:** prorrogar una cadena que ya tenía una prórroga sin aprobar creaba **dos eslabones solapados arrancando el mismo día** — `vigencia_efectiva` solo avanza con las APROBADAS, y la validación de solapamiento no lo veía porque excluye la cadena entera. Ahora la cadena avanza de a un eslabón resuelto por vez. Sin este fix, B3 hacía explotar ese flujo con un `IntegrityError`.

### Consecuencias que quedan vivas:

- **El fallback a sqlite murió:** `DATABASE_URL` es obligatoria y las migraciones solo corren en Postgres (el `ExclusionConstraint` no existe en sqlite). Ya era la regla de `CLAUDE.md`, pero antes el código la toleraba.
- `Novedad.ocupa_periodo` **es un dato denormalizado** (copia del flag del tipo, mantenida en `save()`), porque un `ExclusionConstraint` no puede hacer JOIN a `TipoNovedad`. Si algún día se cambia `ocupa_periodo` en un `TipoNovedad` ya usado, las novedades viejas conservan el valor con el que se cargaron: hay que backfillar a mano.
- **Los locks de concurrencia no tienen test** (una carrera real no es testeable con la infra actual). Para novedades no importa demasiado: el `ExclusionConstraint` es la garantía dura y sí está testeado. Para el legajo no hay red equivalente — si el advisory lock fallara, el UNIQUE tira el mismo error críptico de antes.
- **B4 quedó deliberadamente mínimo:** sin flag `vigente` ni historial de versiones. Renovar un apto médico es mover su `fecha_vencimiento`. La decisión de versionar espera al módulo de documentos con archivos (Drive/OneDrive + metadata en base).

## C. Robustez y rendimiento

---

### C1 — N+1 en la lista de empleados ●

```
gestion_rrhh/apps/empleados/models.py:105-112 +
gestion_rrhh/apps/empleados/api/serializers.py:39
```

El serializer expone `activo`, que ejecuta `self.relaciones.filter(...).exists()`: una query **por empleado** listado, a pesar de que el selector ya prefetchea `relaciones`. Con 25 por página son ~25 queries extra; el patrón se paga también en `relacion_activa`.

**Fix:** resolver sobre el cache prefetcheado (`any(r.estado == EstadoRelacion.ACTIVA for r in self.relaciones.all())`).

### C2 — El dashboard dispara ~50 queries por carga ●

```
gestion_rrhh/apps/dashboard/selectors.py:143-151
```

La serie de rotación de 12 meses llama `_rotacion_periodo` + `_activos_a` por mes (3-4 queries cada una). Funciona, pero escala mal y el panel es la pantalla de entrada.

**Fix:** agregación única por mes (una query con `TruncMonth` para ingresos/egresos y otra para la dotación) o un cache corto (60 s) del dict completo.

### C3 — `getAllPages` ignora errores HTTP ●

```
frontend/integration/ceibo-api.js:90-98
```

No chequea `r.ok`: si una página responde 401/429/500, `d.results` es `undefined` y devuelve silenciosamente una lista parcial o vacía — la UI muestra "0 empleados" sin error. Además `page_size=200` supera el `max_page_size=100` del backend (se clampea: funciona, pero duplica requests sin saberlo).

**Fix:** lanzar error si `!r.ok` (como hace `jget`) y usar `page_size=100`.

### C4 — Throttle compartido entre login y refresh ●

```
gestion_rrhh/apps/usuarios/api/views.py:8-17
```

Ambas vistas usan el scope `login` (5/min). Varias pestañas refrescando a la vez pueden comerse el cupo y cortar sesiones válidas; y un atacante de fuerza bruta se beneficia de que el refresh legítimo comparta su límite.

**Fix:** scope propio para refresh (p. ej. 30/min).

### C5 — Índice compuesto para el chequeo de solapamiento ● (mitigado)

```
gestion_rrhh/apps/novedades/services.py
```

`_validar_sin_solapamiento` filtra por `(empleado, estado__in, ocupa_periodo)` y trae **todas** las candidatas del empleado a Python. Con historial largo por empleado conviene filtrar el rango de fechas en SQL en vez de descartarlo en Python. El índice ya no hace falta: el `ExclusionConstraint` de B3 trae su propio índice GiST sobre `(empleado_id, daterange(...))`, que cubre este filtro.

---

## D. Mejoras funcionales y de proceso

---

### D1 — Workflow de novedades incompleto

`EN_PROCESO` y `CERRADA` existen en el enum (`novedades/models.py:15-23`) pero no tienen endpoint de transición; el front los mapea a "sin acción" (`ceibo-api.js:152-154`). O se implementan (`/tomar/`, `/cerrar/`) o se quitan del contrato hasta que existan.

### D2 — Sin auditoría (RP8)

El TODO está declarado en `novedades/services.py:7-9`: no hay `RegistroAuditoria`. Hoy solo se sabe quién creó (`creado_por`) y quién aprobó (`aprobada_por`); rechazos, anulaciones, ediciones y bajas no registran actor ni momento (el motivo se concatena en `observaciones`, que es frágil). Para RRHH — dominio con disputas legales — es de las mejoras de más valor.

### D3 — Alertas de vencimiento: está el dato, falta el proceso

`DocumentoEmpleado.fecha_vencimiento` está indexado "para la query de alertas", `Parametro` existe para los días de aviso, `Empresa.referente_rrhh` es "el destinatario de avisos"... pero no hay ningún job/endpoint/consumidor que los use. Los tres están muertos hoy. Un endpoint `GET /documentos/por-vencer/` + panel en el dashboard sería el primer paso natural (n8n puede consumirlo después).

### D4 — Sin adjuntos

No hay `FileField` en todo el sistema: certificados médicos y documentos son solo metadata (fechas y números). Está bien para MVP1, pero conviene decidir dónde vivirán los archivos (S3/local) antes de que el modelo crezca.

### D5 — El front carga todo en memoria

`listEmpleados / listNovedades` traen **todas** las páginas y filtran client-side. Con cientos de registros va bien; con miles, la carga inicial y el ranking van a doler. El backend ya soporta filtros y paginación — el front podría usarlos cuando duela (no antes).

### D6 — `getOrCreatePuesto` crea catálogo por texto libre

`ceibo-api.js:248-256`: un typo en "Chofer"/"chofer"/"Choferr" crea puestos duplicados que después ensucian filtros. Mejor un `<select>` con opción "crear nuevo..." explícita.

## D7 — Sin CI

No hay `.github/workflows`. Hay tests y ruff configurados ( `pyproject.toml` ) pero nada los corre automáticamente. Un workflow mínimo (ruff + pytest contra Postgres de servicio + `python frontend/build.py` para validar anclas) protegería justo los puntos frágiles del proyecto.

## D8 — Detalles menores del front

- `anios()` ( `ceibo-api.js:118-121` ): muestra mínimo "1 años" para cualquier antigüedad menor a un año, y "1 años" es gramaticalmente incorrecto → "6 meses" / "1 año".
- `splitName()` ( `ceibo-api.js:113-117` ): "Juan Carlos Pérez" → nombre "Juan", apellido "Carlos Pérez". Heurística inevitable con un solo campo; considerar dos campos en el canvas.
- Novedades filtradas por empresa dependen de `relacion_laboral` no nulo ( `novedades/selectors.py:49-50` ): una novedad sin relación asociada desaparece de ese filtro (el dashboard ya lo resuelve aparte; la lista no).

## Prioridades sugeridas

Toda la sección B salió de esta tabla: está resuelta (2026-07-15).

| Prioridad | ID    | Hallazgo                                              | Esfuerzo          |
|-----------|-------|-------------------------------------------------------|-------------------|
| ● 1       | A1    | Login real; sacar credenciales hardcodeadas           | Medio             |
| ● 2       | A2    | Scope en <code>GET /empleados/{id}/documentos/</code> | Trivial (1 línea) |
| ● 3       | A3    | PII solo para RRHH/Admin                              | Bajo              |
| ● 4       | A4    | <code>SECRET_KEY</code> sin default en prod           | Trivial           |
| ● 5       | C1    | N+1 de <code>activo</code>                            | Trivial           |
| ● 6       | C3    | Errores en <code>getAllPages</code>                   | Trivial           |
| ● 7       | C2    | Dashboard en menos queries                            | Medio             |
| ● 8       | D7    | CI mínima                                             | Bajo              |
| ● 9       | D2    | Auditoría (RP8)                                       | Alto              |
| ● 10      | D3    | Alertas de vencimiento                                | Medio             |
| ● —       | resto | D1, D4, D5, D6, D8, C4, C5                            | según roadmap     |

**Lo próximo, sin vueltas:** A2 es una línea y hoy cualquier usuario con rol Empleado puede leer los documentos de cualquier otra persona. A1 es lo que vuelve real a todo el sistema de roles (hoy todo visitante es admin). Ninguno de los dos se arregló acá.